Cognitive Modeling

Complete Lecture Notes

Week 3: Fitting Models by Least Squares: Error Surfaces and Optimization

Contents

1 Parameter Estimation Methods 1

1.1 Week 3: Fitting Models by Least Squares: Error Surfaces and Optimization 1

1.1.1 Key terms 1

1.1.2 Learning outcomes 2

1.1.3 Motivation & big picture 3

1.1.4 Lecture notes 3

1.1.5 Worked example 10

1.1.6 Lab (2 hours) 11

1.1.7 Connection to AI & LLMs 16

1.1.8 Common mistakes & misconceptions 16

1.1.9 Check your understanding 17

1.1.10 Assignment 18

1.1.11 Further reading 18

Chapter 1

Parameter Estimation Methods

1.1 Week 3: Fitting Models by Least Squares: Error Surfaces and Optimization

Chapter 2: Parameter estimation methods · Lecture 1 h · Lab 2 h This week's running data: the simulated forgetting experiment from Week 2 (power model, $A^* = 0.9$, $d^* = 0.5$, $n = 50$ items at 8 intervals). We refit it properly and compare with last week's linearized fits.

1.1.1 Key terms

Important terms of this lesson, with their Persian equivalents (repeated even if they appeared in earlier weeks).

English	فارسی	توضیح کوتاه
fit / fitting	برازش	تنظیم پارامترها تا مدل با داده جور شود
least squares	کمترین مربعات	روش برازش با کمینه کردن مجموع مربع خطاها
error surface	سطح خطا	خطا به‌عنوان تابعی از پارامترها
optimizer	بهینه‌ساز	الگوریتمی که کمینه‌ی تابع را پیدا می‌کند
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimate / estimator	برآوردگر / برآورد	روش به / مقدار به‌دست‌آمده برای پارامتر دست آوردن آن
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان

English	فارسی	توضیح کوتاه
exponential / power function	توانی / تابع نمایی	دو شکل رایج برای منحنی فراموشی و یادگیری
SSE / RMSD	مجموع مربعات خطا / ریشه‌ی میانگین مربع انحراف	معیارهای فاصله‌ی داده و پیش‌بینی
noise	نویز	تغییرات تصادفی در شواهد یا در داده
parameter trade-off	بده‌بستان پارامترها	تغییر یک پارامتر که با پارامتر دیگر جبران می‌شود
grid approximation	تقریب شبکه‌ای (grid)	محاسبه‌ی پسین روی شبکه‌ای از مقادیر
Nelder–Mead	روش Nelder–Mead	بهینه‌ساز بدون نیاز به مشتق
Bayes factor (BF)	عامل بیز (Bayes factor)	نسبت درست‌نمایی‌های حاشیه‌ای دو مدل
local minimum	کمینه‌ی محلی	نقطه‌ای پایین‌تر از اطراف ولی نه بهترین نقطه
bounds / constraints	قیدها / کران‌ها	محدودیت‌های مجاز برای مقدار پارامترها
reparameterization	بازپارامتری‌سازی	نوشتن پارامتر در مقیاس دیگر، مثلاً $\lambda = e^{\eta}$
multi-start	شروع چندگانه (multi-start)	اجرای بهینه‌ساز از چند نقطه‌ی شروع
maximum likelihood estimate (MLE)	برآورد بیشینه‌ی درست‌نمایی	پارامترهایی که داده را محتمل‌ترین می‌کنند
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
gradient	گرادیان	جهت و شیب تغییر تابع
convergence	همگرایی	رسیدن زنجیره‌ها به توزیع هدف
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
scaling law	قانون مقیاس	رابطه‌ی توانی بین اندازه‌ی مدل و خطا

1.1.2 Learning outcomes

By the end of this week, students will be able to:

1. Define SSE, MSE and RMSD as discrepancy functions and explain what minimizing them assumes about the noise.
2. Visualize an error surface for a two-parameter model and interpret its shape (minimum, valleys, plateaus) in terms of parameter trade-offs.
3. Implement least-squares fitting by grid search and with `scipy.optimize.minimize` (Nelder–Mead and L-BFGS-B), and compare the algorithms' strengths and failure modes.
4. Diagnose and remedy common fitting failures: bad starting values, plateaus, local minima, and out-of-range parameters (using bounds, reparameterization and multi-start).

5. Evaluate a least-squares fit with RMSD and residuals, and interpret the fitted parameters.

1.1.3 Motivation & big picture

Last week we fitted forgetting curves by drawing a straight line through log-transformed proportions. The power model was the true model, yet its estimates were noticeably off ($\hat{A} = 0.80$, $\hat{d} = 0.37$ instead of 0.9 and 0.5). The method also breaks whenever a count is zero, and it minimizes errors on the wrong (log) scale.

The fix is conceptually simple: choose the parameters that make the model's predictions as close as possible to the data, on the data's own scale. "As close as possible" needs a number, a *discrepancy function*. "Choose the parameters" needs a search procedure, an *optimization algorithm*. This week covers both.

There is a deeper reason this matters for cognitive science. Model parameters are our *measurements* of latent constructs (decay rate, caution, sensitivity). A poor fitting procedure gives poor measurements, and every conclusion drawn from them (group differences, correlations with brain activity, model comparisons) inherits that error. Worse, as we will see, an optimizer can report "success" while returning nonsense.

Where is this used?

- Every model-based study in psychology has a fitting step. Learning curves, forgetting curves, psychometric functions (how detection improves with stimulus strength) and dose–response curves in psychopharmacology are all fitted by minimizing a discrepancy.
- Clinical and developmental research compares fitted parameters between groups (e.g. the forgetting rate of patients vs. controls). If the fit is poor or stuck at a bound, the group comparison is wrong.
- Neuroscience fits tuning curves of neurons and hemodynamic response functions in fMRI with exactly these optimizers.
- Machine learning trains models by minimizing a loss with gradient methods; the vocabulary (loss surface, learning rate, local minima) is the same as this week's.

1.1.4 Lecture notes

In plain words A model has knobs (parameters). Fitting means turning the knobs until the model's curve passes as close as possible to the data points. To do this with a computer we need two things. First, a single number that says how bad the current fit is: the sum of squared errors (SSE), the squared vertical distances between data points and the curve, added up. Second, a strategy to turn the knobs so that this number goes down: an optimizer. Think of the SSE as a landscape over all possible

knob settings, with hills (bad fits) and valleys (good fits); the optimizer is a hiker looking for the lowest point. Problems arise when the landscape has a long flat valley (two knobs can compensate each other), a flat plateau (the hiker cannot tell which way is down), or several separate valleys (the hiker may stop in the wrong one). Most of this week is about recognizing those situations and protecting yourself against them.

Suggested timing (60 min): discrepancy functions 10 · error surfaces 10 · optimization algorithms 20 · practical problems and a fitting checklist 15 · preview 5.

1. Discrepancy functions

For observations y_i and model predictions $\hat{y}_i = f(x_i; \boldsymbol{\theta})$, $i = 1, \ldots, N$:

$$\text{SSE}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \big(y_i - f(x_i; \boldsymbol{\theta})\big)^2, \qquad \text{MSE} = \frac{\text{SSE}}{N}, \qquad \text{RMSD} = \sqrt{\text{MSE}}.$$

- All three have the same minimizer: they are monotone transformations of each other. We *minimize* SSE and *report* RMSD, because RMSD is in the units of the data (here: proportion recalled), so "RMSD = 0.057" means predictions are off by about 0.06 on average.
- The least-squares estimate is
$$\hat{\boldsymbol{\theta}}_{\text{LS}} = \arg\min_{\boldsymbol{\theta}} \text{SSE}(\boldsymbol{\theta}).$$

Why squares? Squaring makes all errors positive, penalizes large errors heavily, and gives a smooth (differentiable) function that optimizers handle well. More deeply, minimizing SSE is exactly maximum likelihood estimation if the noise is Gaussian with the same variance for every observation (Week 4). That "if" is the hidden assumption of least squares.

When the assumption fails. Recall proportions are binomial: their variance $f(1-f)/n$ depends on f. Points near 0.5 are noisier than points near 0 or 1, but plain SSE treats them as equally reliable. Weighted least squares fixes this by weighting each squared error by the inverse of its variance:

$$\text{WSSE}(\boldsymbol{\theta}) = \sum_{i} \frac{\big(y_i - \hat{y}_i\big)^2}{\text{Var}(y_i)}.$$

You will try this in Exercise 3. Next week, maximum likelihood handles this issue more naturally.

> Note: some textbooks call RMSD "RMSE". They are the same quantity.

Reading the math

- $\sum_{i=1}^{N}$ means "add up over all N data points". $y_i - f(x_i; \boldsymbol{\theta})$ is the residual of point i: observed minus predicted. Squaring and adding gives one number per parameter setting $\boldsymbol{\theta}$.
- $\arg\min_{\boldsymbol{\theta}} \text{SSE}(\boldsymbol{\theta})$ reads "the value of $\boldsymbol{\theta}$ at which SSE is smallest". Note the difference: min is the smallest *SSE value*, arg min is the *parameter value* that achieves it. We want the arg min.

- Small example. Data $y = (0.62, 0.41)$, predictions $(0.66, 0.44)$: residuals $(-0.04, -0.03)$, SSE $= 0.0016 + 0.0009 = 0.0025$, MSE $= 0.00125$, RMSD $= 0.035$: the curve misses the points by about 3.5 percentage points on average.
- In WSSE each squared residual is divided by the variance of that point, so a noisy point counts less. With binomial data the variance is $f(1-f)/n$.

2. The error surface

For a model with k parameters, SSE($\boldsymbol{\theta}$) is a surface over a k-dimensional parameter space. With $k = 2$ we can draw it as a contour map: the fitting problem becomes "find the lowest point of the landscape."

Features to recognize:

Feature	What it looks like	What it means
Single, round bowl	Concentric, nearly circular contours	Parameters well determined and roughly independent; any decent algorithm finds the minimum
Long, diagonal valley	Elongated contours tilted at an angle	Parameter trade-off: increasing one parameter can be compensated by changing the other. Estimates will be correlated and uncertain along the valley (Week 5, Week 7)
Plateau	Very widely spaced contours	The model's predictions barely change with the parameters (e.g., a decay rate so large that the predicted curve is 0 everywhere). Gradient-based methods stall
Several separate basins	More than one closed set of contours	Local minima: the algorithm's answer depends on where it starts

For exponential forgetting, A and λ trade off: a higher starting level with faster decay can predict nearly the same mid-range values as a lower level with slower decay. The valley is therefore tilted (you will see it in the lab).

3. Optimization algorithms

All algorithms below start from initial values $\boldsymbol{\theta}^{(0)}$ and generate a sequence $\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \ldots$ that (hopefully) descends to the minimum.

3.1 Grid search Evaluate the SSE at every point of a regular grid and keep the best.

- Pros: trivially simple; cannot be fooled by local minima *on the grid*; gives a picture of the whole surface.
- Cons: precision limited by grid spacing; cost grows exponentially with the number of parameters. With 100 values per parameter: $k = 2$ needs 10^4 evaluations, $k = 5$ needs 10^{10}. This is the curse of dimensionality.
- Use it for: visualizing 1–2-parameter surfaces, and for generating good starting values.

3.2 Nelder–Mead simplex (derivative-free) A simplex is a set of $k + 1$ points in a k-dimensional space (a triangle when $k = 2$). Each iteration tries to replace the worst vertex $\boldsymbol{\theta}_w$ by moving it through the centroid $\bar{\boldsymbol{\theta}}$ of the other vertices:

Move	Formula (standard coefficients)	When
Reflect	$\boldsymbol{\theta}_r = \bar{\boldsymbol{\theta}} + (\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}_w)$	Always tried first
Expand	$\boldsymbol{\theta}_e = \bar{\boldsymbol{\theta}} + 2(\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}_w)$	The reflected point is better than the current best; try going further
Contract	$\boldsymbol{\theta}_c = \bar{\boldsymbol{\theta}} + \frac{1}{2}(\boldsymbol{\theta}_w - \bar{\boldsymbol{\theta}})$ (inside), or the same step on the reflected side (outside)	The reflected point is not better than the second-worst vertex
Shrink	Move all vertices halfway toward the best one	Contraction also failed

The reflected point is accepted when it is better than the second-worst vertex but not better than the best. The simplex "tumbles" downhill, stretches along valleys, and shrinks around the minimum.

- Pros: needs only SSE values (no derivatives); robust to noisy or non-smooth objective functions; the default workhorse in cognitive modeling.
- Cons: can be slow in high dimensions; can stall in long narrow valleys; no guarantee of finding the global minimum.

> Direct-search methods like Nelder–Mead are treated in depth in Ghodousian's book on direct methods in nonlinear optimization (see Further reading).

3.3 Gradient-based methods The gradient $\nabla\mathrm{SSE}(\boldsymbol{\theta})$ points uphill, so we step the other way:

$$\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta\,\nabla\mathrm{SSE}\big(\boldsymbol{\theta}^{(m)}\big), \qquad \frac{\partial\,\mathrm{SSE}}{\partial\theta} = -2\sum_i \big(y_i - \hat{y}_i\big)\,\frac{\partial \hat{y}_i}{\partial\theta},$$

with step size (learning rate) η. For the exponential model, $\partial\hat{y}/\partial A = e^{-\lambda t}$ and $\partial\hat{y}/\partial\lambda = -A\,t\,e^{-\lambda t}$.

In practice we use quasi-Newton methods such as BFGS. They also estimate the *curvature* of the surface and choose the step size automatically, so they converge much faster near a minimum. L-BFGS-B is the variant that accepts bounds on parameters. If you do not supply the gradient, scipy approximates it numerically by finite differences.

- Pros: fast and precise on smooth surfaces; scale well to many parameters (this is how neural networks are trained).
- Cons: need a smooth objective; get stuck where the gradient is (nearly) zero, at plateaus as well as at minima; sensitive to starting values.

> Advanced: why plateaus trap gradient methods. If λ is very large, $e^{-\lambda t} \approx 0$ at every observed t, so both partial derivatives above are ≈ 0. A gradient method sees a "flat floor" and stops, sometimes reporting successful convergence. Nelder–Mead compares function values at distant points and can step off the plateau. You will see both behaviors in lab Part D.

Summary:

Method	Needs derivatives	Bounds	Typical use
Grid search	No	Built in	Visualization, starting values, $k \leq 2$
Nelder–Mead	No	Yes (scipy ≥ 1.7)	Default for small cognitive models; noisy objectives
BFGS	Numerical or analytic	No	Smooth objectives, unconstrained parameters
L-BFGS-B	Numerical or analytic	Yes	Smooth objectives with bounded parameters

Reading the math The gradient step $\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta \, \nabla \mathrm{SSE}(\boldsymbol{\theta}^{(m)})$ reads: *new guess = old guess − step size × slope*. The superscript (m) is the iteration number, not a power. ∇SSE is the vector of partial derivatives (one slope per parameter). The minus sign makes the step go downhill.

The derivative formula $\partial \, \mathrm{SSE}/\partial\theta = -2 \sum_i (y_i - \hat{y}_i) \, \partial\hat{y}_i/\partial\theta$ follows from the chain rule. It says: points where the residual is large *and* where the prediction is sensitive to θ pull hardest on θ. At the minimum these pulls cancel and the sum is zero.

For an engineer

- Least squares is the same criterion as in channel estimation and curve fitting; the difference here is that the models are nonlinear in the parameters ($e^{-\lambda t}$), so there is no closed-form

normal-equation solution and we need iterative search.

- Weighted least squares is noise whitening: dividing by the variance is what you do before LS when the noise is not white (heteroscedastic noise).
- Gradient descent with step size η is the steepest-descent idea behind the LMS adaptive filter. BFGS is a quasi-Newton method (it builds an approximation of the Hessian), close to Gauss–Newton / Levenberg–Marquardt used in nonlinear LS.
- A long diagonal valley in the error surface is an ill-conditioned problem: the Hessian has a large eigenvalue ratio, as with strongly correlated regressors. Parameters are well determined along one direction and poorly along the other.
- Reparameterization ($\lambda = e^{\eta}$) is a change of variables that turns a constrained problem into an unconstrained one; it also often improves conditioning.
- What is different from engineering practice: data sets are small (8 points, 50 items each), the noise is large, and the parameters must be *interpreted* psychologically. So it is not enough that the predicted curve looks good: the individual parameter values must be trustworthy.

4. Practical problems and how to handle them

1. Starting values. Use informed starts: read them off a plot, use a quick linearized fit (Week 2), or take the best point of a coarse grid.

2. Local minima $\rightarrow$ multi-start. Run the optimizer from many random starting points and keep the best solution. If many starts agree on the same minimum, you can be reasonably confident it is the global one. Multi-start is cheap insurance; use it routinely for models with more than two parameters.

3. Parameter bounds. Many parameters have natural ranges ($0 \leq A \leq 1$, $\lambda > 0$). Without constraints, optimizers happily return $A = 29.6$. Three approaches:

Approach	How	Comment
Bounded optimizer	`method="L-BFGS-B"` or `"Nelder-Mead"` with `bounds=`	Easiest; recommended default
Reparameterization	Optimize over unconstrained η and transform: $\lambda = e^{\eta}$ (positive), $A = 1/(1 + e^{-\alpha_A})$ (between 0 and 1)	Works with any optimizer; also used in Bayesian modeling (Week 13)
Penalty	Return a huge SSE when parameters are out of range	Creates discontinuities; avoid

4. Non-identifiability. Sometimes *several* parameter combinations fit equally well. An example is a two-component model whose two components can swap labels. This is a property of the model, not a failure of the optimizer, and multi-start reveals it (lab Part D).

5. Trust, but verify. `result.success==True` only means that the algorithm's stopping rule was met. Always (i) plot the fitted curve over the data, (ii) look at the residuals, and (iii) check that the

estimates are within plausible ranges.

A fitting checklist (use it in every lab from now on):

1. Plot the data.
2. Write the model and the discrepancy function; test them on a few hand-picked parameter values.
3. Choose sensible bounds and starting values.
4. Optimize, preferably with multi-start.
5. Check the convergence message, the plot, the residuals and the parameter ranges.
6. Run a recovery check: simulate data from the fitted parameters, refit, and compare (Week 7 makes this systematic).

> Advanced: conditionally linear parameters. In $f(t) = A\,e^{-\lambda t}$, for any fixed λ the best A has a closed form, because the model is linear in A: $\hat{A}(\lambda) = \sum_i y_i g_i / \sum_i g_i^2$ with $g_i = e^{-\lambda t_i}$. Substituting it back gives a one-dimensional problem in λ alone. This trick ("profiling out" linear parameters, or *variable projection*) makes fitting faster and more stable, and the resulting 1-D function is related to the *profile likelihood* of Week 5.

For the instructor

- Show the landscape first. A contour plot of SSE over (A, λ) (lab Part B) makes every later idea concrete: the minimum, the diagonal valley, the plateau. Put it on the projector early and keep coming back to it.
- Act out Nelder–Mead. Draw a triangle on the contour plot and do one reflection by hand (worked example Step 2). Students remember the "tumbling triangle" much better than the table of moves.
- Your strength: you know gradient descent and least squares. Spend less time on the algorithms and more on the *failure modes* (Part D of the lab): an optimizer reporting success at $A = 29.6$ is the most memorable lesson of the week.
- Typical questions.
  - *"Which optimizer should I use?"* For small cognitive models: Nelder–Mead or L-BFGS-B with bounds, always with multi-start.
  - *"Why not always use a grid?"* The cost grows exponentially with the number of parameters.
  - *"Is SSE the right measure?"* Only if noise is Gaussian with equal variance; next week replaces it with the likelihood.

1.1.5 Worked example

Data. A participant recalls proportions $y = (0.62, 0.41, 0.19)$ at $t = (1, 3, 7)$ days. Model: $f(t) = A\,e^{-\lambda t}$.

Step 1: a small grid search by hand

For $(A, \lambda) = (0.8, 0.2)$: predictions $0.8e^{-0.2} = 0.655$, $0.8e^{-0.6} = 0.439$, $0.8e^{-1.4} = 0.197$, so

$$\text{SSE} = (0.62-0.655)^2+(0.41-0.439)^2+(0.19-0.197)^2 = 0.00123+0.00084+0.00005 \approx 0.00212.$$

Repeating this for a 2×3 grid:

	$\lambda = 0.15$	$\lambda = 0.20$	$\lambda = 0.25$
$A = 0.8$	0.02281	0.00212	0.00364
$A = 0.9$	0.06638	0.02172	0.00791

The grid's best point is $(0.8, 0.20)$. Note the tilt: $(0.9, 0.25)$ beats $(0.9, 0.15)$ by far. A larger A "wants" a larger λ, which is the trade-off valley.

Step 2: one Nelder–Mead iteration by hand

Start with the simplex (triangle) of three grid points:

Vertex	(A, λ)	SSE	Rank
$\boldsymbol{\theta}_b$	(0.8, 0.25)	0.00364	best
$\boldsymbol{\theta}_g$	(0.8, 0.15)	0.02281	second-worst
$\boldsymbol{\theta}_w$	(0.9, 0.15)	0.06638	worst

1. Centroid of all vertices except the worst: $\bar{\boldsymbol{\theta}} = \big((0.8 + 0.8)/2,\ (0.25 + 0.15)/2\big) = (0.8, 0.20)$.
2. Reflect the worst vertex through the centroid: $\boldsymbol{\theta}_r = (0.8, 0.20) + \big((0.8, 0.20) - (0.9, 0.15)\big) = (0.7, 0.25)$.
3. Evaluate: predictions $(0.545, 0.331, 0.122)$, so $\text{SSE}(\boldsymbol{\theta}_r) = 0.01657$.
4. Decide: $0.00364 \le 0.01657 < 0.02281$. The reflected point is better than the second-worst but not better than the best, so we accept the reflection (no expansion).
5. New simplex: $\{(0.8, 0.25),\ (0.7, 0.25),\ (0.8, 0.15)\}$, and the worst SSE has dropped from 0.066 to 0.023.

Step 3: let the computer finish

Running `scipy.optimize.minimize(...,method="Nelder-Mead")` from $(0.8, 0.2)$ converges to $\hat{A} = 0.754$, $\hat{\lambda} = 0.200$, with SSE $= 3.7 \times 10^{-5}$ (RMSD $= 0.0035$).

Interpretation. The fitted model says this participant would recall about 75% immediately after learning and loses about 18% of the remaining material per day ($1 - e^{-0.2} = 0.181$), with a half-life of $\ln 2/0.2 \approx 3.5$ days. With only three data points and two parameters, the near-perfect fit says very little about whether the exponential model is *right*. Any reasonable two-parameter decreasing curve would fit three points well (Week 6).

1.1.6 Lab (2 hours)

Goals

1. Write an SSE function and explore the error surface of two forgetting models.
2. Fit by grid search, then with Nelder–Mead and L-BFGS-B; compare with last week's linearized fits.
3. Provoke and fix fitting failures: plateaus, local minima and out-of-range estimates.
4. Evaluate the fits (RMSD, residuals, comparison with the true values) and interpret the parameters.

Timing: Part A 15 min · Part B 25 min · Part C 25 min · Part D 35 min · Part E 20 min.

Setup and data

Code · کد: `code/week03.py`, cell 1 · سلول 1

Part A: Get a feel for the discrepancy function (15 min)

Code · کد: `code/week03.py`, cell 2 · سلول 2

Expected output:

```
theta = (0.9, 0.5):  power SSE = 0.0349   exponential SSE = 0.4511
theta = (0.8, 0.4):  power SSE = 0.0328   exponential SSE = 0.4145
theta = (0.9, 0.1):  power SSE = 0.9047   exponential SSE = 0.2192
theta = (0.6, 0.05):  power SSE = 0.3303   exponential SSE = 0.1145
```

Before running the next parts, predict: which parameter values will give the smallest SSE for each model? Write your guess down.

Part B: Grid search and the error surface (25 min)

Code · کد: `code/week03.py`, cell 3 · سلول 3

Expected output:

```
exp: grid minimum A = 0.655, rate = 0.0560, SSE = 0.10607  (42300
↪  evaluations)
pow: grid minimum A = 0.870, rate = 0.4239, SSE = 0.02592  (42300
↪  evaluations)
```

What to notice: both surfaces have a single basin, but it is an elongated, tilted valley: higher A goes with higher decay. That is the parameter trade-off. For the power model the true value (white ×) lies inside the valley but not at its bottom: with finite data, the best fit is never exactly the truth.

Part C: Fitting with an optimizer (25 min)

Code · کد: `code/week03.py`, cell 4 · سلول 4

Expected output:

```
model            method      A   rate    SSE   RMSD  f_evals success
  exp          L-BFGS-B 0.6568 0.0566 0.1061 0.1151     39.0    True
  exp       Nelder-Mead 0.6568 0.0566 0.1061 0.1151     69.0    True
  exp              grid 0.6550 0.0560 0.1061 0.1151  42300.0    True
  exp linearized (Wk 2) 0.5500 0.0320 0.1373 0.1310      NaN     NaN
  pow          L-BFGS-B 0.8712 0.4251 0.0259 0.0569     36.0    True
  pow       Nelder-Mead 0.8711 0.4251 0.0259 0.0569     71.0    True
  pow              grid 0.8700 0.4239 0.0259 0.0569  42300.0    True
  pow linearized (Wk 2) 0.7970 0.3700 0.0315 0.0628      NaN     NaN
```

Code · کد: `code/week03.py`, cell 5 · سلول 5

What to notice: - Nelder–Mead and L-BFGS-B agree to three or four decimals; the grid is within its spacing (0.005 for A) of them. The optimizers needed only 36–71 function evaluations instead of 42 300. - Least squares gives a lower SSE than the linearized fits of Week 2 (it is minimizing exactly that quantity), and the power estimates are much closer to the truth. - The path first heads quickly into the valley and then crawls along it, a typical pattern.

Part D: When fitting goes wrong (35 min)

D1. A plateau and a false "success". Start the exponential fit with an absurd decay rate, $\lambda = 20$ per day:

Code · کد: `code/week03.py`, cell 6 · سلول 6

Expected output:

```
BFGS         A =   29.571  lambda =  14.244  SSE = 1.1616  success =
 ↪  True  (Optimization terminated successfully.)
L-BFGS-B     A =   29.905  lambda =  14.289  SSE = 1.1616  success =
 ↪  True  (CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL)
Nelder-Mead  A =    0.657  lambda =   0.057  SSE = 0.1061  success =
 ↪  True  (Optimization terminated successfully.)
```

(The wording of the L-BFGS-B message varies slightly between SciPy versions.) The two gradient-based methods report success but return a nonsensical fit: A is around 30 (a recall probability of 3000%!) and the predicted curve is essentially zero at every interval. They started on a plateau where the gradient is almost zero (Lecture §3.3, Advanced box). Nelder–Mead escaped. Lessons: (1) always check estimates and plots, not just `success`; (2) use bounds; (3) use sensible or multiple starting values.

D2. Reparameterization. Make the parameters unconstrained so that *any* optimizer respects the ranges $0 < A < 1$ and $\lambda > 0$:

Code · کد: `code/week03.py`, cell 7 · سلول 7

Expected output:

```
BFGS on reparameterized power model: A = 0.8712, d = 0.4251, SSE =
 ↪  0.02592
```

Same answer as in Part C, and the estimates are guaranteed to lie in range.

D3. Local minima and multi-start. A popular idea in memory research is that forgetting reflects two memory traces: a fast component that decays quickly and a slow one that lasts:

$$f(t) = A_1 e^{-\lambda_1 t} + A_2 e^{-\lambda_2 t}.$$

This model has $k = 4$ parameters, and its error surface has several basins. Fit it from 50 random starting points:

Code · کد: `code/week03.py`, cell 8 · سلول 8

Expected output:

```
Distinct final SSE values (rounded) and how often each was reached:
  SSE = 0.0201: 42 starts
  SSE = 0.0203: 1 starts
  SSE = 0.0337: 1 starts
  SSE = 0.0340: 1 starts
  SSE = 0.0359: 1 starts
  SSE = 0.1064: 1 starts
  SSE = 0.2010: 1 starts
  SSE = 0.2456: 1 starts
  SSE = 0.9092: 1 starts
Best 3 solutions (A1, lam1, A2, lam2):
   [0.58  0.36  0.264 0.   ] SSE = 0.02013
   [0.264 0.    0.58  0.36 ] SSE = 0.02013
   [0.264 0.    0.58  0.36 ] SSE = 0.02013
```

What to notice: - About 40 of 50 starts end in the same best basin, but the rest (about 1 in 6) get stuck in worse local minima, some with an SSE many times larger. (The exact counts differ slightly between SciPy versions; with SciPy 1.18 it is 41 of 50.) A single run from an unlucky start would have given a wrong answer, and no warning. - Among the best solutions, the fast and slow components may appear with labels swapped ($A_1, \lambda_1 \leftrightarrow A_2, \lambda_2$). Both describe the same curve. This is a non-identifiability built into the model, and a common fix is to impose an order constraint such as $\lambda_1 > \lambda_2$. - In the best solution one component has a rate of exactly 0 (a flat, non-decaying component of 0.264). The fitted two-trace model has therefore collapsed into the exponential-with-asymptote model of Exercise 4, with the same SSE (0.0201). Is that a meaningful "permanent memory" or just the extra parameters absorbing noise? Remember that the data came from a *power* model. Fit alone cannot tell (Weeks 6–7).

Part E: Evaluate and interpret (20 min)

Code · کد: `code/week03.py`, cell 9 · سلول 9

Expected output:

```
Power model: relative loss rate at t =  1 days: 0.213 per day
Power model: relative loss rate at t = 20 days: 0.020 per day
Exponential model: constant relative loss rate 0.057 per day
```

Interpret (write 2–3 sentences for each in your notebook):

1. Compare the residual patterns. Which model shows a systematic pattern, and what does that pattern say about the shape of forgetting in these data?
2. The power model's relative loss rate falls from about 0.21 per day after 1 day to about 0.02 per day after 20 days. Explain in plain language what this claims about memory.
3. The least-squares estimates ($\hat{A} \approx 0.87$, $\hat{d} \approx 0.43$) are closer to the truth than last week's linearized estimates but still not equal to it. Is this a failure of the method? (Hint: what would happen if you reran the experiment with a new random sample?)

Exercises (increasing difficulty)

Exercise 1 $\star$ (same minimizer). Fit the power model by minimizing SSE, MSE and RMSD in turn. Show that the estimates agree (up to optimizer tolerance). Why do we still prefer to *report* RMSD?

Exercise 2 $\star$ (Ebbinghaus by least squares). Refit the Ebbinghaus savings data from Week 2, Exercise 3 (values marked [verify]) with least squares, for both models. Compare the parameters and RMSD with your linearized fits from Week 2.

Exercise 3 $\star\star$ (weighted least squares). Binomial proportions have variance $f(1-f)/n$. Implement WSSE (Lecture §1) using the predicted f in the variance, refit the power model and compare the estimates with plain least squares. Which data points gain influence and which lose it? (Next week you will see that maximum likelihood does this weighting automatically.)

Exercise 4 $\star\star$ (more parameters, better fit?). Fit the three-parameter exponential-with-asymptote model $f(t) = B + (A - B)e^{-\lambda t}$, where B is the long-term asymptote, using bounds $0 \le B \le A \le 1$ (hint: reparameterize, or check the constraint after fitting). *Expected:* SSE ≈ 0.020, lower than the power model's ≈ 0.026, even though the data were generated by the power model! What does this tell you about using SSE to choose between models with different numbers of parameters? (We answer this properly in Weeks 6–7.)

Exercise 5 $\star\star\star$ (Advanced: Nelder–Mead from scratch). Implement Nelder–Mead for $k = 2$ using the moves in Lecture §3.2 (reflect, expand, inside/outside contract, shrink) and a stopping rule based on the spread of SSE values in the simplex. Run it on the power model from $(0.8, 0.2)$, plot the simplex every few iterations on the error surface, and compare the result and number of function evaluations with `scipy`.

Lab check-off

Show the instructor or TA: (1) the error-surface plots with grid minima (Part B), (2) the comparison table from Part C, (3) your multi-start histogram (Part D3) with one sentence on what it shows, and (4) your answer to interpretation question 1 (Part E).

1.1.7 Connection to AI & LLMs

Scaling laws are curve fitting. One of the most influential results in modern AI is that the test loss of language models falls smoothly with model size N, data size and compute, following a power law with an asymptote: $L(N) = E + B\,N^{-\alpha}$ (Kaplan et al., 2020; Hoffmann et al., 2022). Companies use such fits to decide how big the next model should be. Mathematically, this is the same problem as fitting a forgetting curve: a nonlinear model with three parameters, fitted by least squares with a numerical optimizer and multiple starting points. It has the same pitfalls too: extrapolation far beyond the data, and trade-offs between parameters (here E and α).

Code · کد: `code/week03.py`, cell 10 · سلول 10

Expected output:

```
E = 1.70, B = 3.22, alpha = 0.358, SSE = 0.00424
Extrapolated loss at N = 1e11: 1.754
```

Try it (no API needed). - ⋆ Fit a pure power law without the asymptote ($E = 0$). Compare the SSE and the extrapolation to $N = 10^{11}$. Which conclusion about "bigger is better" changes? - ⋆⋆ Fit only the four smallest models and extrapolate to the four largest. How far off is the prediction? Repeat with different seeds to see how stable the extrapolation is. - ⋆⋆ Plot the SSE surface over E and α (with B at its best value for each pair). Is there a ridge, as in the Week 3 lab?

Optional reading: Kaplan, J., et al. (2020). Scaling laws for neural language models. *arXiv:2001.08361*. Hoffmann, J., et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.

1.1.8 Common mistakes & misconceptions

1. "**`success=True`** means the fit is good." It only means a stopping rule was met (Part D1). Always inspect estimates, plots and residuals.
2. One start is enough. For anything beyond simple two-parameter models, run multi-start. The cost is small; the risk of reporting a local minimum is not.
3. Ignoring parameter ranges. Unbounded optimizers can return impossible values (negative rates, probabilities above 1). Use bounds or reparameterization.
4. Comparing SSE across models with different numbers of parameters. The more flexible model almost always wins on SSE (Exercise 4). That is a question of model *evaluation*, not estimation (Weeks 6–7).
5. Comparing SSE across datasets of different sizes. SSE grows with N; use RMSD for comparisons.

6. Treating the minimizer as the true value. $\hat{\boldsymbol{\theta}}$ is an estimate with sampling error. A different sample would give a different estimate (Week 5 quantifies this).
7. Forgetting what least squares assumes. Plain SSE assumes equal, Gaussian noise for all observations. Proportions, counts and RTs usually violate this.
8. Using grid search for many parameters. The cost explodes (100^k evaluations). Use grids only for visualization and starting values.

1.1.9 Check your understanding

1. Why do SSE, MSE and RMSD have the same minimizer? Which one would you report, and why? Answer MSE = SSE/N and RMSD = $\sqrt{}$MSE are strictly increasing transformations of SSE (N is fixed), so the same parameters minimize all three. Report RMSD, because it is in the units of the data and comparable across datasets of different sizes.

2. What is a parameter trade-off, and how does it appear on an error surface? Answer Changes in one parameter can be compensated by changes in another, leaving the predictions (and the SSE) almost unchanged. On the surface it appears as an elongated, tilted valley of low SSE. Estimates are then correlated and poorly determined along the valley direction.

3. With 50 grid values per parameter, how many SSE evaluations does a grid search need for a model with 6 parameters? What does this imply? Answer $50^6 \approx 1.56 \times 10^{10}$ evaluations. Grid search is infeasible beyond 2–3 parameters (the curse of dimensionality); we need iterative optimizers.

4. In one Nelder–Mead iteration, the reflected point is better than the current best vertex. What does the algorithm do next? Answer It tries an expansion: a point twice as far from the centroid in the same direction. It keeps the expanded point if it is better than the reflected point, otherwise the reflected point.

5. Why can a gradient-based optimizer stop on a plateau and report success? Answer Its stopping rule is based on the gradient (or step size) being nearly zero. On a plateau the gradient is close to zero even though the point is not a minimum, so the convergence criterion is met.

6. Give two ways to keep a decay-rate parameter positive during fitting. Answer (i) Use a bounded optimizer (e.g., L-BFGS-B or Nelder–Mead with `bounds=[(1e-6,None)]`). (ii) Reparameterize: optimize η on the whole real line and set $\lambda = e^{\eta}$.

7. Multi-start fits of the two-component model reach the same minimum SSE with $(A_1, \lambda_1, A_2, \lambda_2) = (0.58, 0.36, 0.26, 0.00)$ and $(0.26, 0.00, 0.58, 0.36)$. Is this a problem with the optimizer? Answer No. It is label switching: swapping the two components gives the identical curve, so the model is not identifiable without an ordering constraint (e.g., $\lambda_1 > \lambda_2$). The optimizer correctly found two equivalent global minima.

1.1.10 Assignment

No new graded assignment is released this week. A1 (From verbal to formal) is due at the start of the Week 4 lecture. Today's lab gives you everything needed for A1's calibration step if you want to check your closed-form derivation numerically.

Optional practice (ungraded, recommended before Week 4): complete lab Exercises 3 and 4. Exercise 3 (weighted least squares) is the bridge to maximum likelihood, and Exercise 4 previews the central problem of model evaluation. Solutions will be discussed briefly at the start of the Week 4 lab.

1.1.11 Further reading

- Farrell & Lewandowsky (2018), chapter 3. Basic parameter estimation: discrepancy functions, error surfaces, the simplex algorithm, and practical issues such as local minima and starting values. *(Main reading.)*
- Ghodousian (1401), *Mabani-ye ravesh-haye mostaghim dar behine-sazi gheyr-khatti* (in Persian): direct search methods for nonlinear optimization, including simplex-type methods. Recommended for students who want the theory behind Nelder–Mead [verify chapter].
- Verguts (2022): the chapter on optimization and gradient descent, useful for the gradient-based view that returns in Week 15 (learning models) [verify chapter].

Preview for Week 4: instead of asking "which parameters minimize squared error?", we ask "which parameters make the observed data most probable?" This is maximum likelihood. It handles binomial data correctly, and it is the foundation for AIC/BIC (Week 7) and Bayesian inference (Weeks 9–13).